

A deep understanding of AI large language model mechanisms				
Section name	Video lecture title	Python code file name	Udemy video #	Key take-home points
Introductions	[IMPORTANT] Prerequisites and how to succeed in this course	-	1	LLM architecture, training, and mechanisms are advanced topics. Have a positive attitude and embrace the challenge.
	Using the Udemy platform	-	2	-
	Getting the course code, and the detailed overview	-	3	-
	Do you need a Colab Pro subscription?	-	4	You can access GPUs for free on Google Colab, though compute time and RAM are limited.
	About the "CodeChallenge" videos	-	5	You are the master of your education, and you should engage with this course in a way that best suits your skills and goals.
Part 1: Tokenizations and embeddings				
Words to tokens to numbers	Why text needs to be numbered	-	7	Text must be transformed into numbers before LLM.
	Parsing text to numbered tokens	part1_text2num_text2numbers	8	Text can be split into words via spaces, although this is not done in real tokenization.
	CodeChallenge: Create and visualize tokens (part 1)	part1_text2num_CTokenizeTokenizer	9	Encoders and decoders are created using dictionary comprehension and functions.
	CodeChallenge: Create and visualize tokens (part 2)	part1_text2num_CTokenizeTokenizer	10	"One-hot encoding" is a sparse tokenization, with one row per token and one column per vocab item.
	Preparing text for tokenization	part1_text2num_preparingTextTokens	11	Real text from the web is easy to import but a pain to clean...
	CodeChallenge: Tokenizing The Time Machine	part1_text2num_CCTimeMachineTokens	12	Randomly generated tokens are usually nonsensical.
	Tokenizing characters vs. subwords vs. words	-	13	Every tokenization scheme has advantages and limitations.
	Byte-pair encoding algorithm	part1_text2num_bytePairEncoding	14	The BPE algorithm is based on replacing frequent token sequences with new tokens.
	CodeChallenge: Byte-pair encoding to a desired vocab size	part1_text2num_CBytePairEncodingLoop	15	Even simple byte-pair encoding on a tiny dataset creates tokens with preceding spaces, just like professional tokenizers.
	Exploring ChatGPT's tokenizer	part1_text2num_GPT4tokenizer	16	OpenAI's tokenizer is available, but is model-specific (e.g., GPT2 vs. GPT4).
	CodeChallenge: Token count by subword length (part 1)	part1_text2num_CTokenEfficiency	17	Words and tokens differ in several ways, though they can overlap.
	CodeChallenge: Token count by subword length (part 2)	part1_text2num_CTokenEfficiency	18	Words and tokens differ in several ways, though they can overlap.
	How many "y"s in strawberry?	part1_text2num_strawberry	19	ChatGPT has difficulties with letter-based calculations because it represents words as tokens.
	CodeChallenge: Create your algorithmic rapper name :)	part1_text2num_CCrapperName	20	Tokenization can be fun!
	Tokenization in BERT	part1_text2num_BERT	21	Different tokenizers are optimized for different purposes.
	CodeChallenge: Character counts in BERT tokens	part1_text2num_CCharChars	22	Professionals tokenizers are easy to work with once you get used to them.
	Translating between tokenizers	part1_text2num_tokenTranslation	23	Write your code for one specific tokenizer.
	CodeChallenge: More on token translation	part1_text2num_CCTranslatorFuns	24	The more familiar you are with tokenization, the easier it will be to understand embeddings and LLM mechanisms.
	CodeChallenge: Tokenization compression ratios	part1_text2num_CTokenCompression	25	Importing text data from the web is really easy.
	Tokenization in different languages	part1_text2num_languages	26	Tokenization = compression (but it often is).
	CodeChallenge: Zipf's law in characters and tokens	part1_text2num_CZipf	27	Zipf's law, a.k.a. power-law scaling, a.k.a. scale-free organization, a.k.a. self-similarity is a pervasive characteristic of biological and physical systems, and is taken as evidence of complex systems.
	Word variations in Claude tokenizer	part1_text2num_CClaudeVariations	28	Spaces are meaningful to humans, but are treated just like any other character to tokenizers.
	Word2Vec vs. GloVe vs. GPT vs. BERT... oh my!	-	29	There are several word embeddings matrices that have different goals and applications, and are created in different ways. It is not trivial to compare them (more on this in the mech.interp section). The embeddings used in LLMs are not fixed, but instead are adjusted by the model based on context.
Embedding spaces	Exploring GloVe pretrained embeddings	part1_embed_GloVe	30	The GloVe embeddings can be used to study texts and relations between words.
	CodeChallenge: Wikipedia vs. Twitter embeddings (part 1)	part1_embed_CCoWkTwitter	31	A lot of diversity across word embeddings matrices.
	CodeChallenge: Wikipedia vs. Twitter embeddings (part 2)	part1_embed_CCoWkTwitter	32	A lot of diversity across word embeddings matrices.
	Exploring GPT2 and BERT embeddings	part1_embed_GPT2BERT	33	All model parameters of publicly available LLMs are easily accessible.
	CodeChallenge: Math with tokens and embeddings	part1_embed_CCoMathWithTokens	34	Token indices are arbitrarily mapped to meaningful subwords (numbers).
	Cosine similarity (and relation to correlation)	part1_embed_cosineSimilarity	35	Cosine similarity is one of the most commonly used measures of a relationship between two variables when working with embeddings and LLMs.
	CodeChallenge: GPT2 cosine similarities	part1_embed_CCosineSimilaritiesGPT2	36	Multitoken words present several challenges in LLM investigations (hint: give the model context and only analyze the final token).
	CodeChallenge: Unembeddings (vectors to tokens)	part1_embed_CCoUnembedding	37	An "unembeddings matrix" is the exactual inverse of the embeddings matrix (but not a literal inverse matrix).
	Position embeddings	part1_embed_positionEmbeddings	38	Language models use position embeddings to focus on tokens from various locations ("time points") in the token sequence.
	CodeChallenge: Exploring position embeddings	part1_embed_CCPositionExplorations	39	The position embeddings matrix is complicated and impacts token processing in ways that are difficult to predict a priori.
	Training embeddings from scratch	-	40	Embeddings matrices in LLMs are trained with the rest of the model. In the next several videos, however, we will train only and embeddings layer, to focus on the mechanisms of learning embeddings.
	Create a data loader to train a model	part1_embed_learnEmbeddings	41	Preparing data to train models can become complicated (a common experience in many data fields...).
	Build a function to learn the embeddings	part1_embed_learnEmbeddings	42	Embeddings matrices are learned from text data using gradient descent and dimension-squeezing deep learning models.
	Loss function to train the embeddings	part1_embed_lossFunction	43	Negative log likelihood is the main loss function used in language model training.
	Train and evaluate the model	part1_embed_learnEmbeddings	44	Language models are trained on a small number of epochs, because each epoch has a huge amount of data (batches).
Part 2: Large language models	CodeChallenge: How the embeddings change	part1_embed_learnEmbeddings	45	The embeddings vectors expanded to fill up more of the embeddings space, reflecting encoding of high-dimensional information in a lower-dimensional space.
	CodeChallenge: How stable are embeddings?	part1_embed_learnEmbeddings	46	Embeddings vectors of the same token appear to be completely unrelated across repeated training runs.
Build a GPT	Why build when you can download?	-	48	Building a model from scratch is a fantastic way to learn how LLMs are created and trained.
	Model 1: Embedding (input) and unembedding (output)	part2_build_model1	49	Saying that language models "use only the final token" for next-token prediction is not accurate. They use all tokens; the final token contains the most information.
	Understanding nn.Embedding and nn.Linear	part2_build_embeddingVlinear	50	nn.Embedding is a convenient wrapper for nn.Parameter. Use it to create embeddings matrices.
	CodeChallenge: GELU vs. ReLU	part2_build_CGeluVgelu	51	GELU is the most common activation function in LLMs. It is more complicated than ReLU but smoother.
	Softmax (and temperature): math, numpy, and pytorch	part2_build_softmax	52	Softmax transforms LLM outputs from "raw" values (logits) into a probability distribution.
	Randomly sampling words with torch.multinomial	part2_build_multinomial	53	Torch.multinomial directly maps token probability values onto selection probabilities.
	Other token sampling methods: greedy, top-k, and top-p	-	54	There is no right or wrong token selection method.
	CodeChallenge: More softmax explorations	part2_build_CCSoftmaxExtreme	55	Softmax is mathematically simple, but some aspects of the transformation are apparent only for some numerical ranges.
	What, why, when, and how to layernorm	part2_build_layernorm	56	Layernorm is simple and critical.
	Model 2: Position embedding, layernorm, tied output, temperature	part2_build_model2	57	Position embeddings are added to the token embeddings, and help the model learn temporal patterns.
	Temporal causality via linear algebra (theory)	-	58	Time-causal attention can be implemented using a time vector in which the future is weighted zero while the past is weighted non-zero.
	Averaging the past while ignoring the future (code)	part2_build_pastWithInlag	59	Matrix multiplication with masks are an efficient way to avoid for-loops.
	The "attention" algorithm (theory)	-	60	The "attention" mechanism is a clever way of pooling information across different embeddings vectors that allows for surrounding context to modify the current token transformation.
	CodeChallenge: Code Attention manually and in Pytorch	part2_build_CAttentionAlgo	61	Q, K, and V matrices are trainable weights that are not changed during inference (applications).
	Model 3: One attention head	part2_build_model3	62	Attention adjusts (not replaces) the embeddings vectors as they pass through the model.
	The Transformer block (theory)	-	63	The Transformer block contains an attention sublayer and an MLP sublayer. Together, they calculate an adjustment to the token embedding to point towards an appropriate next-token embedding.
	The Transformer block (code)	part2_build_transformer	64	You can now implement a GPT-style Transformer :)
	Model 4: Multiple Transformer blocks	part2_build_model4	65	Use nn.Sequential to create repeated instances of the same component of a model (with different weights matrices).
	Multithread attention: theory and implementation	part2_build_multithreadAttention	66	Multithread Attention (MHA) involves applying the attention equation to different slices of the QKV matrices.
	Working on the GPU	part2_build_GPU	67	GPUs are great at number-crunching, and can save a lot of time in LLM training and applications.
	Model 5: Complete GPT2 on the GPU	part2_build_model5	68	We just built a GPT2-small :) although the weights are all random, so it's not functional.
	CodeChallenge: Time models on CPU and GPU	part2_build_CGpuVsCpu	69	LLMs are basically worthless without a large number of dedicated high-end GPUs.
	Inspecting OpenAI's GPT2	part2_build_OpenAIGPT2	70	Publicly available pretrained models are really easy to use, explore, experiment with, and learn from.
Pretrain LLMs	Summarizing GPT using equations	-	71	There are many ways to understand a deep learning model; a combination of perspectives (diagrams, explanations, code, equations) is most beneficial.
	Visualizing nano-GPT	https://bbystroft.net/llm	72	A neat website where you can visualize different GPT variants.
	CodeChallenge: How many parameters? (part 1)	part2_build_CParameterCounts	73	Biases are a tiny fraction of model parameters, which is why many deep learning models (especially models with layernorm) ignore them.
	CodeChallenge: How many parameters? (part 2)	part2_build_CParameterCounts	74	Biases are a tiny fraction of model parameters, which is why many deep learning models (especially models with layernorm) ignore them.
	CodeChallenge: GPT2 trained weights distributions	part2_build_CCoWeightsDists	75	Characteristics of model weights often have smooth transitions across layers, reflecting shifts in representations and calculations.
	CodeChallenge: Do we really need Q?	part2_build_CClabotomizeQ	76	Causal manipulations are easy to implement, although knowing what to manipulate is challenging and non-trivial.
	What is "pretaining" and is it necessary?	-	77	Model weights are not random. They are mostly smooth transitions across layers, but many are not random. They are mostly smooth transitions across layers, but many are not random.
	Introducing huggingface	-	78	Pretaining is a necessary first step for any LLM. A pretrained model understands the structure and patterns of written language, and can generate text.
	The AdamW optimizer	-	79	HuggingFace provides resources for downloading pretrained LLMs (and other models), training datasets, and more. We will use some of the free resources in this course. You do not need a HuggingFace login to access materials for this course.
	CodeChallenge: SGD vs. Adam vs. AdamW	part2_pretrain_CSGdVsAdams	80	L2 regularization in Adam involves updating and regularizing in one step (i.e., regularizing the update).
	Train model 1	part2_pretrain_model1	81	Simple mechanisms work better in simple models.
	CodeChallenge: Add a test set	part2_pretrain_CModel1Test	82	Code to train has the same basic organization as code to train any deep learning model.
	CodeChallenge: Train model 1 with GPT2's embeddings	part2_pretrain_CModel1WithEmbeds	83	Creating and evaluating a test set is not so difficult, but requires extra code.
	CodeChallenge: Train model 5 with modifications	part2_pretrain_model5WithMods	84	Freezing weights is a common technique in deep learning for transfer learning or when re-training on limited data.
	Create a custom loss function	part2_pretrain_customLoss	85	There are several ways to sample data, depending on how meticulous you want the procedure. Some overlap or skipping is less consequential with limitless training data.
	CodeChallenge: Train a model to like "X"	part2_pretrain_CCTrainXbias	86	Loss functions are extremely important for training deep learning models.
	CodeChallenge: Numerical scaling issues in DL models	part2_pretrain_CScaling	87	KL divergence is a useful loss function for training distributions instead of individual tokens.
	Weight initialization	part2_pretrain_weightInits	88	Each matrix multiplication (dot products) increases the variance and numerical range of numbers.
	CodeChallenge: Train model 5 with weight inits	part2_pretrain_CModel5WeightInits	89	Weight initialization is not important in small models, but is crucial for training large models including LLMs.
	Dropout in theory and in Pytorch	part2_pretrain_dropout	90	Now you know how to initialize weights :)
	Should you output logits or log-softmax(logits)?	-	91	Dropout involves "switching off" units during training, and is thought to promote distributed representations.
	The FineWeb dataset	part2_pretrain_FineWeb	92	Calculating log-softmax inside the LLM is often fine and convenient during training and classification, but outputting the raw values allows for more flexibility in subsequent applications.
	CodeChallenge: Fine dropout in model 5 (part 1)	part2_pretrain_CModel5dropout	93	HuggingFace provides several high-quality datasets for LLM training, including FineWeb (and several more specific variants).
	CodeChallenge: Fine dropout in model 5 (part 2)	part2_pretrain_CModel5dropout	94	Dropout is conceptually simple, but adding it to code can be tricky.
	CodeChallenge: What happens to unused tokens?	part2_pretrain_CUnusedTokens	95	Dropout is conceptually simple, but adding it to code can be tricky.
				Because softmax affects all token logits, not just the ones currently being processed, all tokens embeddings are trained inside the model, not just the ones that appear in the current token sequence.

A deep understanding of AI large language model mechanisms				
Section name	Video lecture title	Python code file name	Udemy video #	Key take-home points
Fine-tune pretrained models	Optimization options	-	96	There are many ways to decrease computation time during LLM training.
	What does "fine-tuning" mean?	-	97	Fine-tuning means making targeted adjustments to an existing pretrained model. It takes days or weeks instead of months, and has much smaller computational requirements.
	Fine-tune a pretrained GPT2	part2_finetune_GPT2gulliver	98	Shortcuts are great, but make sure you understand the mechanisms before oversimplifying your code.
	CodeChallenge: Gulliver's learning rates	part2_finetune_CClearningRateGulliver	99	Smaller learning rates are generally preferred during fine-tuning.
	On generating text from pretrained models	part2_finetune_Hfgenerate	100	The Huggingface tokenizers and pretrained LLMs have many options for encoding and generating text. Some warning messages can be ignored, depending on your goal.
	CodeChallenge: Maximizing the "x" factor	part2_finetune_CChowxT	101	Fine-tuning can be tricky, because it is easy to make a good base model be less useful.
	Alice in Wonderland and Edgar Allen Poe (with GPT-neo)	part2_finetune_2models2styles	102	There are many pretrained models to use and explore. Different models have slightly different architecture, different parameters, and different training datasets, based on the specific goals of the org.
	CodeChallenge: Quantify the Alice/Edgar fine-tuning	part2_finetune_CC2modelsQuantify	103	It would be great to rely entirely on quantitative evaluation scores, but for generative models, numbers don't capture the success of training or the application value.
	CodeChallenge: A chat between Alice and Edgar	part2_finetune_CCconvAliceEdgar	104	Concatenating token sequences, generating new tokens, and displaying only the newly generated tokens is a simplified implementation of chatbots.
	Partial fine-tuning by freezing attention weights	part2_finetune_freezeAttention	105	Freezing trainable parameters is toggled by the boolean requires_grad property of a tensor.
	CodeChallenge: Fine-tuning and targeted freezing (part 1)	part2_finetune_CCFreezingNeo	106	Precision Freezing isn't so difficult to implement; knowing which weights to freeze and why is challenging and uncertain.
	CodeChallenge: Fine-tuning and targeted freezing (part 2)	part2_finetune_CCFreezingNeo	107	Precision Freezing isn't so difficult to implement; knowing which weights to freeze and why is challenging and uncertain.
	Parameter-efficient fine-tuning (PEFT)	-	108	PEFT is a set of related techniques that allow you to fine-tune models with limited computational resources.
	CodeGen for code completion	part2_finetune_CodeGen	109	Always look for specialized pretrained models! They often come in multiple sizes (parameter counts).
	CodeChallenge: Fine-tune codeGen for calculus	part2_finetune_CcCodeGenCalc	110	To an LLM, any token sequence is just a token sequence; it is humans who make the distinction between spoken languages, code languages, time series, music, etc.
	Fine-tuning BERT for classification	part2_finetune_bert4classification	111	Base models are very versatile, and you can use them in many ways.
	CodeChallenge: IMDB sentiment analysis using BERT	part2_finetune_CCBertClassifier	112	Language models are very versatile and can be incorporated into myriad tasks that are based on processing text, code, or any other temporal sequence that can be tokenized.
	Gradient clipping and learning rate scheduler (part 1)	part2_finetune_clipScheduler	113	Gradient clipping involves scaling down, without fundamentally changing, the gradient magnitude. It slows and stabilizes learning.
	CodeChallenge: Clip, freeze, and schedule BERT	part2_finetune_CCBertClipScheduler	114	Each additional step of LLM training is straightforward, but together they can present and overwhelming number of choices and decisions.
	Saving and loading trained models	part2_finetune_saveAndLoadModels	116	There are several ways to save and load models, depending on the environment and subsequent goals when re-importing.
	BERT decides: Alice or Edgar?	part2_finetune_bertWhichAuthor	117	Training language models to classify text is easy — and easier to evaluate compared to generative models.
	CodeChallenge: Evolution of Alice and Edgar (part 1)	part2_finetune_CCBertClassifiesAliceEdgar	118	Some seemingly subjective evaluations can be made quantitative by using a separate trained classifier model.
	CodeChallenge: Evolution of Alice and Edgar (part 2)	part2_finetune_CCBertClassifiesAliceEdgar	119	Some seemingly subjective evaluations can be made quantitative by using a separate trained classifier model.
	Why fine-tune when you can use AGP?	-	120	SOTA commercial models are powerful, but cannot be downloaded or fine-tuned.
	Instruction tuning	-	121	Instruction tuning is the same procedure as fine-tuning (which is the same as pretraining). The main difference is the style and content of the datasets.
	Some datasets for instruction tuning	part2_instruct_datasets	122	Instruction tuning datasets look like a desired chat.
	Training a chatbot with system-user-assistant	-	123	Yet again: Many ways of training LLMs looks the same: Predict the next token in a sequence.
	Instruction tuning with GPT2	part2_instruct_GPT2	124	Training datasets for chatbots comprise human-written Q&A texts. The LLM learns to emulate the style and tone.
	CodeChallenge: Instruction tuning GPT2-large (part 1)	part2_instruct_CCinstructingGPT2large	125	LLMs train on such a large number of token sequences that imposing structure that helps us (humans) is not necessarily beneficial to the model.
	CodeChallenge: Instruction tuning GPT2-large (part 2)	part2_instruct_CCinstructingGPT2large	126	LLMs train on such a large number of token sequences that imposing structure that helps us (humans) is not necessarily beneficial to the model.
	Reinforcement learning from human feedback (RLHF)	-	127	RLHF is designed to align models to human values and human-like behavior, including helpfulness and truthfulness.
Part 3: Evaluating LLMs				
Quantitative evaluations	Promises and challenges of quantitative evaluations	-	129	Quantitative evals have the goal of assessing basic and advanced language capabilities and world-knowledge.
	Numerical issues in logits and softmax	part3_evals_numericalIssuesLogits	130	Always check your intermediate and final results carefully, and be mindful of numerical issues.
	Perplexity	part3_evals_perplexity	131	Perplexity is a simple measure of token prediction that can be calculated without access to model internals (need only final output logits).
	CodeChallenge: Perplexing perplexities	part3_evals_CcPerplexities	132	Token logits in pretrained models do not follow a uniform distribution.
	Masked word prediction accuracy	part3_evals_maskedPredictionsBERT	133	Token prediction accuracy measures semantic and grammatical capabilities of LLMs.
	HellaSwag	part3_evals_hellaSwag	134	Sentence-level evaluations have advantages over token-level evals, including the ability to assess world knowledge and reasoning.
	Import large models using bitsandbytes	part3_evals_bitsandbytes	135	You can use the bitsandbytes library to import and use lightweight versions of large models.
	CodeChallenge: HellaSwag evals in two models (part 1)	part3_evals_CChellaModels	136	The success of the attention mechanism has led to increasing varieties of the basic GPT architecture.
	CodeChallenge: HellaSwag evals in two models (part 2)	part3_evals_CChellaModels	137	The success of the attention mechanism has led to increasing varieties of the basic GPT architecture.
	KL (Kullback-Leibler) divergence	part3_evals_KLDivergence	138	KL divergence is a measure of the distance between two probability distributions.
	MAUVE	part3_evals_mauve	139	The MAUVE score ranges between 0 and 1, and reflects how much LLM-generated text "looks like" (has similar distribution characteristics to) human-written text.
	CodeChallenge: Large and small MAUVE explorations	part3_evals_CCmauveVariability	140	A general statistical principle in science is to average multiple noisy measurements in hopes of more accurately assessing characteristics and capabilities.
	SuperGLUE and other amalgamations	-	141	That the diversity and imperfections of individual evaluation methods can be countered by aggregating results across many evals.
	Assessing bias and fairness	-	142	LLMs can exhibit biases against certain groups.
	Non-technical benchmarks	-	143	Statistical measures of task performance are useful, but there are many other ways to think about the impact of LLMs from commercial and culture perspectives.
Qualitative evaluations	Black box evals	-	144	Black box evaluations are important and have revealed many serious safety risks of LLMs.
	Red-teaming	-	145	"Red-teaming" refers to trained security experts using adversarial attacks to discover security, privacy, or safety risks.
	Accuracy, coherence, and relevance	-	146	Some qualities of language are subjective, yet are desired features of a language model.
	Distributions of hidden-state activations	part3_evals_variousVisualizations	147	The internals of the models can be extracted and explored. This is key to mechanistic interpretability and technical AI safety.
	Heatmaps of tokens for qualitative inspection	part3_evals_textHeatmaps	148	Text heatmaps are compelling visualization tools that can facilitate understanding how LLM calculations relate to tokens.
	CodeChallenge: Visualize single-token predictions	part3_evals_CcHeatmapTokenPredictions	149	The final output logits of token N predict token N+1.
Part 4: Overview of AI safety and mechninterp				
AI safety	AI safety and alignment	-	151	AI safety involves technical and legal measures to guide AI development towards maximal benefits with minimal risk.
	Why can't AI just be safe and moral?	-	152	LLMs understand human morality and ethics as token sequence patterns, just like they understand how to write LinkedIn posts or any other pattern.
	In-context and few-shot learning	-	153	ICL is the remarkable ability for LLMs to learn and accurately perform novel tasks without adjusting weights or fine-tuning.
	Scaling and AI safety	-	154	Scaling laws describe the past but cannot guarantee the future.
	Hands-on: Hack an AI to steal a password	-	155	A fun little demo where you try to convince an LLM to give you a secret password.
Interpretability	How to get involved in AI safety	-	156	AI safety is increasingly important, but lags far behind AI development.
	What is "mech interp?" (mechanistic interpretability?)	-	157	Mechanistic interpretability is the attempt to understand LLM mechanisms by reverse-engineering.
	How does mech interp relate to AI safety?	-	158	Mechanistic interpretability will lead to a more precise understanding of LLMs and AI.
	Concepts and terms in mech interp	-	159	Terminology is an important part of learning a new area of knowledge.
	Theoretical and empirical approaches in research and teaching	-	160	Both theory and data analysis are crucial for development of LLM understanding and AI safety.
Part 5: Observation (non-causal) mech interp				
Investigating token embeddings (part 1)	General criticisms of mechanistic interpretability	-	161	There are several valid criticisms of mechanistic interpretability.
	CodeChallenge: Cosine similarity (advanced) (part 1)	part5_embeddings_CCosineSimilarity	163	Pytorch functions take some getting used to...
	CodeChallenge: Cosine similarity (advanced) (part 2)	part5_embeddings_CCosineSimilarity	164	Pytorch functions take some getting used to...
	CodeChallenge: Cosine similarity in word sequences	part5_embeddings_CcsequentialCosines	165	Studying relationships in embeddings matrices can be interesting and thought-provoking, but static embeddings cannot capture the re-interpretations of words that is necessary for realistic communication.
	CodeChallenge: Coloring cosine similarity	part5_embeddings_CcheatmapsCosim	166	antic meaning tend to have shorter embeddings vectors. Shorter vectors are less impactful in dot product calculations.
	CodeChallenge: Can random embeddings be interpreted?	part5_embeddings_CCinterpRandomEmbeds	167	Interpreting patterns in noise is dangerously easy. Generalizing from patterns in small datasets is one of the most important functions of the nervous system...
	TSNE projection and DBSCAN clustering (theory)	-	168	T-SNE and clustering can be insightful methods to group and characterize high-dimensional data.
	TSNE projection and DBSCAN clustering (Python)	part5_embeddings_TSNEclust	169	T-SNE and clustering can be insightful methods to group and characterize high-dimensional data.
	CodeChallenge: cluster the "x" terms	part5_embeddings_CCSINE	170	Selecting internal model representations by linguistic category or feature is easy and can be interesting.
	CodeChallenge: Tokenize, embed, and cluster happy emojis	part5_embeddings_CCemoji	171	Emojis are multitoen characters, and have embeddings just like any other token.
	RSA (representational similarity analysis)	part5_embeddings_RSA	172	RSA allows you to compare representations of tokens in different models, layers, or measurements (e.g., LLM vs. human brain).
	CodeChallenge: Compare embeddings with RSA (part 1)	part5_embeddings_CCembeddingsRSA	173	Your toolkit of analysis methods is growing :)
	CodeChallenge: Compare embeddings with RSA (part 2)	part5_embeddings_CCembeddingsRSA	174	Your toolkit of analysis methods is growing :)
	CodeChallenge: Word2vec vs. GPT2	part5_embeddings_CCword2vecVsGpt2	175	Cosine similarity is sensitive to mean offsets, which is a desired feature in some situations but can be a confound in other situations.
	CodeChallenge: Graph representation of cosine similarities	part5_embeddings_CCGraphs	176	Similarity metrics can be visualized in multiple ways.
	Embeddings arithmetic and analogies	part5_embeddings_analogyvectors	177	Embeddings vectors are just vectors; any geometric or linear-algebra operation can be implemented.
	CodeChallenge: soft-coded analogies in word2vec	part5_embeddings_CCSoftAnalogies	178	Language models are fun to work with :)
	Creating and interpreting linear "semantic axes"	part5_embeddings_semanticAxesNorm	179	Normalization, regularization, selection, and other (often post-hoc) techniques are often necessary to get satisfactory results.
	kNN for synonym-searching in BERT	part5_embeddings_knnBERT	180	kNN (k-nearest neighbor) is a simple and versatile algorithm that can be used for classification or nuclear sampling.
	CodeChallenge: BERT v GPT kNN competition	part5_embeddings_CCKnnBERTGPT	181	Many analyses in machine-learning and linear algebra can be applied to many types of data and for many hypotheses and explorations.
	Research on translating embeddings spaces	-	182	There is active and ongoing research on translating between embeddings vectors in different models.
	Single vector spectrum of embeddings submatrices	part5_embeddings_SVDspectrum	183	SVD is an advanced method for analyzing embeddings (e.g., PCA) but also requires more linear algebra background to fully appreciate.
	CodeChallenge: SVD projections of related embeddings	part5_embeddings_CCSVDprojections	184	SVD, like all other analyses of complex data, can be insightful or confusing (or both), and often raises more questions than answers.
	Activation maximization via gradient ascent (theory)	-	185	Activation maximization is a technique to discover features that activate individual neurons in a DL model.
	Activation maximization (code)	part5_neurons_activationMaximization	186	If you can specify a loss function, you can optimize it using gradient descent.
	Activation maximization via data sampling	part5_neurons_actMaxSampling	187	Data sampling for activation maximization is conceptually simple and likely to be interpretable.
	CodeChallenge: Reproducibility of activation maximization	part5_neurons_CCoactMaxRepeat	188	Individual dimensions in an LLM don't seem to be "tuned" to specific tokens.
	Extracting activations using "hooks"	part5_neurons_PytorchHooks	189	"Hooks" are functions you can implant into a PyTorch model, that allow you to access its activations during a forward pass.

